Static Analysis for Bounds-Check Elision in the Naga WGSL Compiler

Bhuvan B
ACM SIGPLAN-M Mentee, Open Source Contributor
bhuvanb1408@gmail.com

Abstract

WebGPU requires that a shader never reads or writes outside its resources, so the Naga compiler used by wgpu and Firefox inserts a runtime clamp on array indices unless it can prove the index in range. Due to Naga's conservative approach to its implementation, there is no analysis which proves these index ranges. The approach here is an interval abstract interpretation over Naga's IR that removes a clamp only when a proof succeeds, using workgroup-size facts, branch refinement, differences between related values, bit masks, loop recurrences and runtime array lengths. Because the analysis sits on a browser's security boundary and is not mechanised. In this CFP we report what the analysis does, the current state of work, and what the talk will explore, including allowing people to understand WebGPU and the compiler/PL work that goes into vendor-agnostic WebGPU implementations.

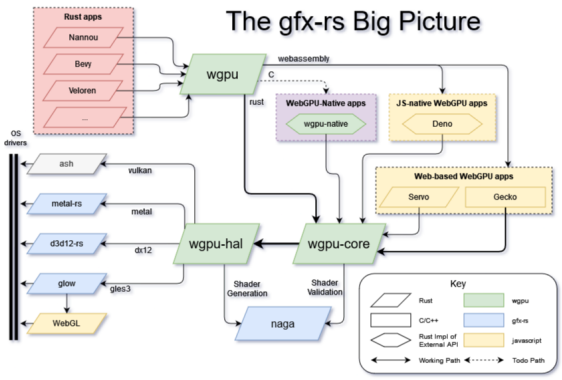
1. Introduction

WGSL spec forbids an out-of-bounds access. Naga's `Restrict` policy rewrites `a[i]` to `a[min(i, len-1)]` for every indexing access . On Vulkan the hardware can bound accesses to buffers, but workgroup and private arrays still need the clamp, whereas on Metal, all accesses get a clamp. A needless clamp costs time and performance, a wrong proof is a memory-safety hole behind a browser sandbox. A missed proof is slow, a false proof is unsafe, but a true proof can help bring out performance by removing this overhead.

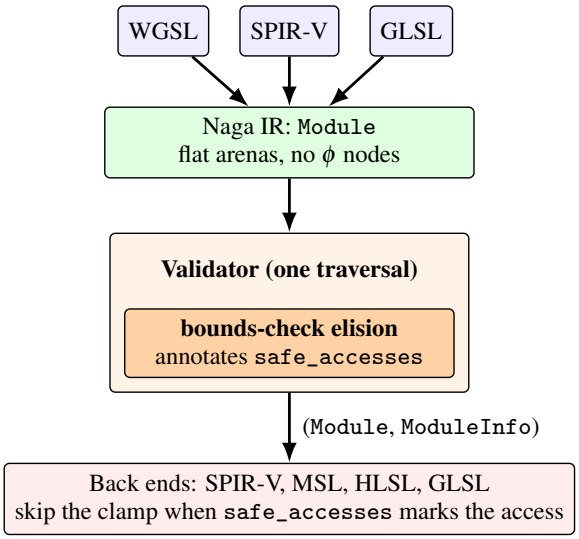
The cost is recorded.In gpuweb, the open issue 1202 (Nov 2020) a 1024×1024 matrix multiply ran 17% to 40% slower with robust buffer access on D3D12 and Metal, the clamp sitting on `acc[clamp(i,0,3)][clamp(j,0,3)]` inside loops guarded by `i < 4`. The working group's resolution (1 Dec 2020) was that Tint needs "some basic range analysis".

2. The Naga Compiler and Where This Pass Fits

Naga is the shader translator in wgpu, the Rust implementation of WebGPU used by Firefox, Servo and Deno. wgpu hands it WGSL source which Naga parses that into a single in-memory module, checks it, and emits the target shading language. Its primary role is shader translation and validation rather than general-purpose optimization to reduce overheads for browser tasks where latency is very important, and that role shapes where this analysis can live. Figure 1(a) shows the position of Naga in the gfx-rs stack: applications drive `wgpu-core`, which call Naga to generate and validate shader code before a back end (`wgpu-hal`) reaches the driver.



(a) Naga in the gfx-rs stack



(b) Naga's one-pass pipeline

Figure 1. Left: Naga in the gfx-rs stack. Right: Naga's pipeline, with bounds-check elision fused into the validator and consumed by the back ends.

Figure 1(b) shows Naga's own flow, which has three stages. A front end (`wgsl`, `spv`, `glsl`) parses source into Naga's IR: a single `Module` holding arenas of types, global variables, constants, functions and, per function, flat arenas of expressions and statements. Naga does not represent functions as an SSA control-flow graph with ϕ nodes; rather, expressions are referenced by handles, while mutable local state is represented by `LocalVariable` like pointers, with storage access for the variables through explicit `Load` and `Store` operations. The IR does not provide the canonical

SSA and loop-analysis infrastructure assumed by analyses such as LLVM's ScalarEvolution. Naga does not expose an LLVM-style general-purpose pass-manager pipeline for repeatedly scheduling analyses and transformations over the IR. Validation is organised around a `Validator` traversal of module and function state, after which the individual back ends (`spv`, `msl`, `hlsl`, `glsl`, `wgsl`) consume the validated IR.

Because there is no general-purpose optimization pipeline, the analysis is not a standalone IR-to-IR pass waiting in a pass list. It is fused into validation: at the end of checking each function, the validator runs the elision analysis with the proof information it has just built, and the analysis records the accesses it has proved safe. Each back end consults that record when deciding whether to emit a clamp, so a proven access reaches code generation without one. It runs once per function per validation and is enabled by default (`set_bounds_check_elision`). The analysis must derive every fact it needs on the fly, during the validation traversal, because no independent SSA or mem2reg step is available to it: intervals, relations and loop summaries are all computed inline. And when a workgroup size is an override, the front end has only stored a placeholder of 1, so builtin ranges are unknown until `process_overrides` resolves the value and the module is re-validated, which re-runs the whole analysis with the real size.

3. Interval Analysis for Index Safety

3.1 Intervals and Overflow-Checked Arithmetic

Every integer variable holds with it its interval information . If this can be proven for index accessing conditions it is accepted as a contributing factor else is considered as *no fact*. Addition, subtraction and multiplication use checked arithmetic on both endpoints and return *no fact* on overflow, so wrap-around never yields a wrong proof. Masks and shifts bound results even from unknown inputs (`x & 7u` is in $[0, 7]$). *no fact* always defaults to a clamp.

3.2 Branch Refinement and Relations

Branches narrow intervals on the taken edge. A relation `big - small` rescues subtractions such as `lid - off` under `if (lid >= off)`.

3.3 Loop Recurrences and Runtime-Sized Arrays

A loop variable matching a known recurrence (shrinking `i >>= c`, growing `i *= f`, or a linear step) gets a header interval from a closed form. A runtime-sized array access needs a dominating `i < arrayLength` on the same array.

3.4 Worked Example: Tree Reduction

With `@workgroup_size(256)` and `var<workgroup> s: array<u32, 256>`:

```
for (var stride = 128u; stride > 0u; stride = stride >> 1u) {
  if (lid < stride) { s[lid] = s[lid] + s[lid + stride]; } }
```

(1) `lid` is in $[0, 255]$ from the workgroup size. (2) `stride` is in $[1, 128]$ because its only update is `» 1`. (3) The guard gives `lid` in $[0, 127]$. (4) So `lid + stride` is in $[1, 255]$, inside the array. All the four clamps Naga emits for this kernel are removed. Information not handled yet,: calls, atomics and `switch` reset what is known about locals; builtin ranges exist only for the local invocation ids with a literal workgroup size; loop recurrences are matched by pattern, as in Go's `parseIndVar`, not computed by a general scalar-evolution engine as in LLVM.

4. What Is Specific to WGSL and Naga IR

Exploration and research discovered four things in Naga IR that change the analysis from a generalized textbook approach . (1) Naga representation is SSA like but with caviates, local variables as storage locations that are mutable. A `LocalVariable` expresses the information as a pointer to that storage location, and any read/write is separated using `LOAD` and `STORE` operations. Therefore, a range fact derived from a load cannot automatically be reused after a store to the same local; the analysis must account for the intervening mutation. (2) Loops have a `continuing` block reached by `continue` edges as well as fall-through, so it must start from the join of all of them. (3) Before overrides are resolved the front end stores a placeholder workgroup size of 1, so builtin ranges are unknown until the overrides are resolved and the module re-validated. (4) WGSL integer rules shape the operators: `%` keeps the sign of the dividend, shift amounts are taken modulo 32, and arithmetic that could wrap returns *no fact*. The initial facts, such as `local_invocation_index` in $[0, \text{workgroup size})$, come from the specification, not from the data flow.

5. Performance Evaluation

5.1 Real-World Clamp Census

Clamps removed on real-world shaders (189 entry points from six projects and wgpu's examples, base variants only). The surviving egui and iced clamps index constant arrays by `vertex_index` and are necessary.

5.2 Microkernels

Five kernels are measured. Table 1 lists what each one does, the clamps it carried, and the speedup on both GPUs; Figure 2 shows the same effect with the spread over runs.

How to read the numbers: every entry in the table says how much performance the analysis adds, measured by running the same kernel twice on the same GPU, once with Naga's clamps and once with them removed. A value of `1.16×` on `scan_shared` means removing the clamps made the kernel run 1.16 times as fast as the clamped build. Entries too small to separate from the measurement noise are marked n.r. and count as neither a gain nor a loss.

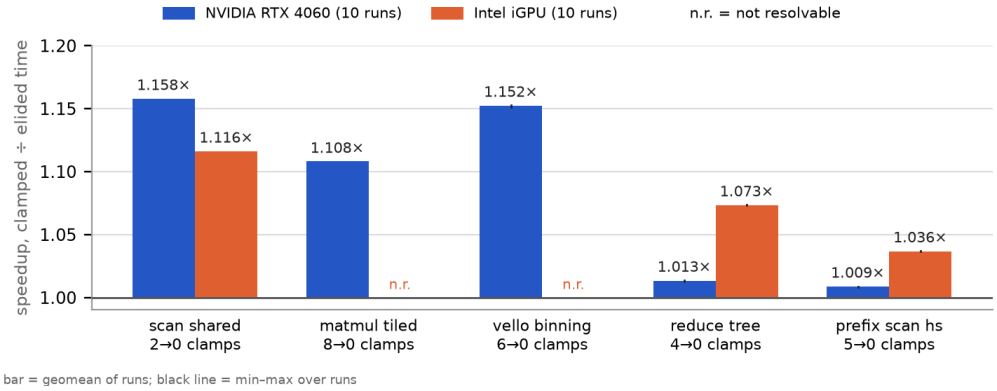


Figure 2. Speedup from elision per microkernel, clamped time over elided time. Bar: geometric mean over runs; black line: min–max over runs. n.r. = not resolvable.

Kernel	What it does	Clamps	NVIDIA	Intel iGPU
`scan_shared`	scan over a 256-element tile	2 to 0	1.16×	1.12×
`vello_binning`	binning, 8×256 atomic bitmaps	6 to 0	1.15×	n.r. (runs disagree in sign)
`matmul_tiled`	16×16 tiled matrix multiply	8 to 0	1.11×	n.r. (control noise $\geq 0.5\%$)
`reduce_tree`	tree reduction, 256 elements	4 to 0	1.01×	1.07×
`prefix_scan_hs`	prefix scan, 256 elements	5 to 0	1.01×	1.04×

Table 1. Per-kernel results. The elided SPIR-V is byte-identical to the `Unchecked` build for every kernel, so each effect comes from the removed clamps.

We only call a result meaningful when it is bigger than the measurement noise; otherwise we mark it **"not resolvable" (n.r.)**. The Intel matmul result was too noisy, and vello's runs disagreed in sign, so neither counts as a real gain or loss.

5.3 Real-World Workloads

The same measurement on 36 workloads: 32 vello scenes and four blur variants (Figure 3). By the pre-registered rule 24 are gains, 7 equivalent and 5 losses. Only 13 lie outside the measured noise floor of about 0.29%: 12 gains (the largest +5.45%, +4.40%, +2.15% and +1.20%) and one loss (−0.59%, `image luminance mask`). The geometric mean over the 32 vello scenes is +0.59% (interval 0.15% to 1.03%). Though the average improvement of the pass is marginal, These tests show , the elision has real measurable increase in real world shaders and improving our approach over the next few weeks will allow to handle the regression cases as well.

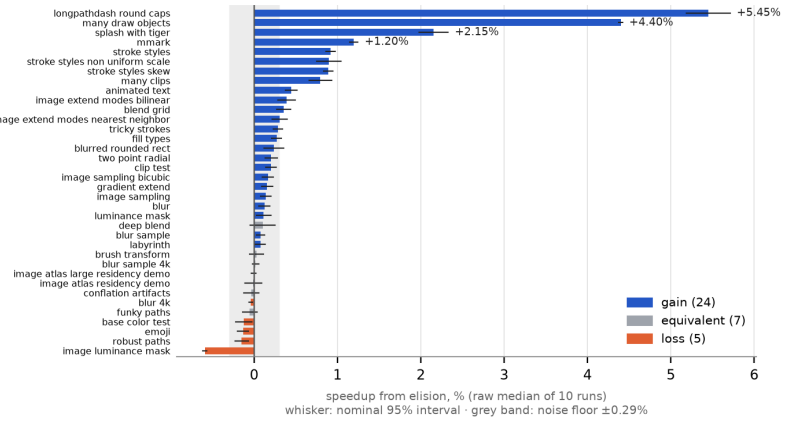


Figure 3. Speedup from elision for every real-world workload on the NVIDIA GPU, sorted. Whisker: nominal 95% interval; grey band: measured noise floor, ±0.29%.

6. Relation to Prior Work

ABCD removes checks on demand over an inequality graph (Bodik et al., PLDI 2000). The HotSpot client compiler does it for Java, later extended to cooperate with deoptimization (Wuerthinger et al., 2007, 2009). LLVM's IRCE splits a loop's iteration space into three segments so the check is redundant in one of them. Go's `prove` pass keeps a signed and an unsigned interval per value and rejects arithmetic whose endpoints can overflow, and its `loopbce` pass recognises induction variables syntactically as `(Phi min nxt)`. Cousot and Cousot (1977) supply the lattice and fixpoint framing. (Sources checked against archived copies; Go pinned at commit 9834516.)

Our application and approach is closest to Go's: per-value intervals, overflow-rejecting endpoint arithmetic, and loop recurrences matched by pattern. It is not ABCD's demand-driven graph and does no loop splitting; relation facts and runtime-length proofs are added on top. The setting is the real difference. In a Java VM a failed check throws and the VM can deoptimise. In WGSL the check is mandatory and its failure behaviour is a clamp the program silently receives, so a wrong proof is an undetected out-of-bounds access, not an exception. Upstream Naga proves only constant indices in bounds (`access_needs_check`); Chrome's Tint was asked in 2020 to add range analysis (gpuweb#1202), and we have not verified what it implements today. Readyset's removal of 651 bounds checks changed nothing beyond "measurement noise" on a CPU; here the checks are mandated and the cost depends on the GPU. The nearest comparison for trusting a range analysis in an untrusted-code compiler is the Linux eBPF verifier, whose range analysis has been machine-checked and found unsound operators (reference [9]).

7. Current Status and Future Work

The framework for the analysis and proof of its improvements are presented with future work includes handling the regression cases, uplifting device information from the GPU to the IR level and using the per device information to perform analysis based on these device state information. Expand the benchmarking, for DirectX and Metal with more test suites.

References

[1] P. Cousot, R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs. POPL 1977, pp. 238–252. link

[2] R. Bodik, R. Gupta, V. Sarkar. ABCD: eliminating array bounds checks on demand. PLDI 2000, pp. 321–333. link

[3] T. Wuerthinger, C. Wimmer, H. Moessenboeck. Array bounds check elimination for the Java HotSpot client compiler. PPPJ 2007; and its extension in the context of deoptimization, 2009.

[4] S. Das. Inductive range check elimination. LLVM (Phabricator D6693).

[5] Go compiler: `prove.go` (per-value `Limit`, `SafeAdd`) and `loopbce.go` (`parseIndVar`), commit 9834516. LLVM `ScalarEvolution`.

[6] W3C. WebGPU and WebGPU Shading Language (WGSL) specifications (WebGPU; WGSL).

[7] gfx-rs/wgpu. Naga, the shader translation library for wgpu (front ends WGSL/SPIR-V/GLSL; back ends SPIR-V/MSL/HLSL/GLSL/WGSL). Source at `naga/src`; stack figure `docs/big-picture.png`.

[8] gpuweb/gpuweb#1202, "Out-of-bounds clamping brings big perf regression on TFJS ML demos", 2020.

[9] gfx-rs/wgpu#4972, #6518, #6662 (Metal infinite loops and forced loop bounding), 2024. Readyset, bounds-check removal write-up, 2022.

[10] "Verifying the Verifier: eBPF Range Analysis Verification" (Springer, doi 10.1007/978-3-031-37709-9_12); "Fixing Latent Unsound Abstract Operators in the eBPF Verifier of the Linux Kernel" (SAS 2024 preprint); "Testing Static Analyses for Precision and Soundness" (CGO 2020). Verify venue and year before citing.

[11] Bhuvan B. Static analysis for bounds check elision in the Naga WGSL compiler. IICT 2026, paper 55 (poster and extended report).